AlphaLab

An End-to-End MLOps Platform for
Quantitative Factor Research on NSE Stocks

Project Report

Submitted in partial fulfilment of the requirements for

B.Tech / B.E. Program

Submitted by

Swara Patil

Indian Institute of Technology Madras (IITM)

April 28, 2026

Abstract

AlphaLab is a full-stack MLOps platform I built to apply machine learning to quantitative equity research on Indian stock markets. The core idea was simple: instead of picking stocks by gut feeling, can we systematically identify factors that predict future returns and build a portfolio around them?

The project covers the entire ML lifecycle — from automated daily data ingestion using Apache Airflow, through factor engineering and XGBoost model training tracked in MLflow, to a REST API served via FastAPI and a React dashboard that lets users explore factor performance, portfolio recommendations, and stock-level explanations. The infrastructure is fully containerised using Docker Compose, and the system is monitored in near-real-time using Prometheus and Grafana.

Across five weeks of development, I implemented 21 quantitative factors across four categories (momentum, value, volatility, and mean reversion), trained a composite ranking model, built a model registry, and set up a monitoring stack with drift detection baselines and alert rules. The result is a production-grade research platform that a quant analyst could realistically use on a day-to-day basis.

Contents

1. Introduction

1.1 Motivation

I wanted to explore the field of quantitative finance and understand how people actually use data to make decisions in markets.

While learning, I came across the idea of factors — simple measurable properties of stocks, like whether a stock is cheap, whether its price has been going up recently, or how risky it is. What I found interesting was that these patterns are not random — they have been observed again and again across markets.

This made me curious to dig deeper: Can I build a system that actually uses these factors to analyze stocks and make predictions?

At the same time, I didn't want this to be just another notebook-based project. I wanted to understand how such systems work in the real world — where data comes in regularly, models are updated automatically, and everything runs in a structured pipeline.

So this project is my attempt to explore both sides — the quant ideas behind factor investing and the engineering required to make it work end-to-end.

1.2 Objectives

The main objectives of this project were:

- Build an automated pipeline that ingests daily OHLCV data for 100 NSE-listed stocks using `yfinance` and Airflow
- Engineer 21 quantitative factor features across four categories and version them with DVC
- Train an XGBoost ranking model and track all experiments in MLflow
- Expose results through a FastAPI REST API with health, readiness, and Prometheus

metrics endpoints

- Build a React frontend with three tabs: factor leaderboard, factor deep-dive, and portfolio view
- Set up Prometheus + Grafana monitoring with alert rules for error rates, latency, and drift
- Containerise the entire stack using Docker Compose

2. System Architecture

2.1 High-Level Overview

The architecture follows a layered MLOps design with clear separation between data ingestion, feature engineering, model training, serving, and monitoring. Figure 2.1 shows the full system.

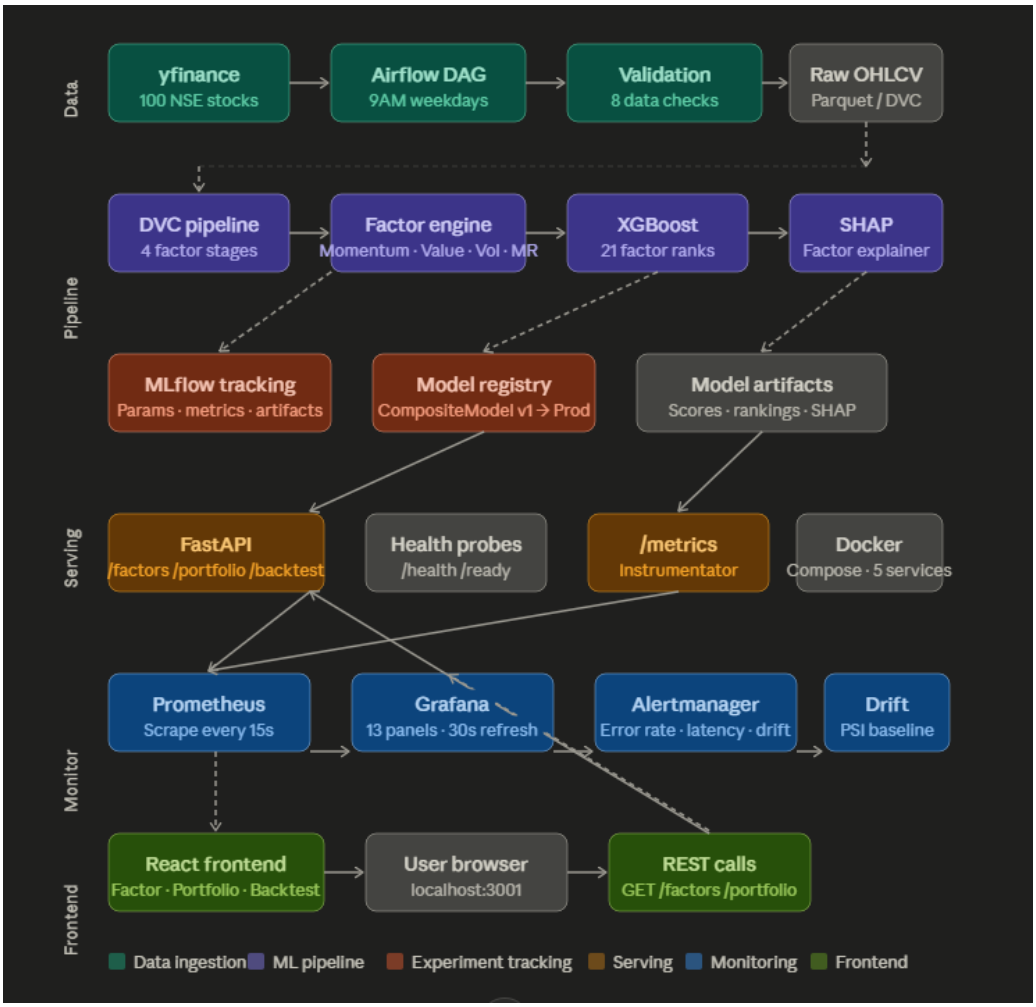

Figure 2.1: AlphaLab end-to-end architecture diagram. Boxes are colour-coded by layer: teal = data ingestion, purple = ML pipeline, coral = experiment tracking, amber = model serving, blue = monitoring, green = frontend.

The system is divided into six layers:

1. **Data ingestion** — Airflow DAG triggers at 9AM on weekdays, pulls OHLCV data for 100 stocks via `yfinance`, runs 8 validation checks, and stores results as Parquet files versioned with DVC.
2. **Feature engineering** — A DVC pipeline runs four factor computation stages (momentum, value, volatility, mean reversion) and combines them into a single feature matrix with 21 columns.
3. **Model training** — XGBoost trains on the 21 factor rank columns; all hyperparameters, metrics, and model artifacts are logged to MLflow. The best model is registered as `AlphaLab_CompositeModel v1` and promoted to Production.
4. **Model serving** — FastAPI loads all Parquet artifacts at startup and serves them through REST endpoints. The API is containerised and exposes a `/metrics` endpoint for Prometheus scraping.
5. **Monitoring** — Prometheus scrapes the API every 15 seconds. Grafana displays a 13-panel dashboard. Alertmanager fires on error rate $> 5\%$, p95 latency > 500ms, and factor drift.
6. **Frontend** — A React application provides three views: the factor leaderboard, factor deep-dive with P&L charts, and the portfolio recommendation view with SHAP explanations.

2.2 Docker Compose Services

The entire backend stack runs as five Docker Compose services:

Table 2.1: Docker Compose services in AlphaLab

Service	Image	Port	Purpose
`api`	Custom (Python 3.12)	8000	FastAPI model server
`prometheus`	prom/prometheus	9090	Metrics collection
`grafana`	grafana/grafana	3000	Metrics visualisation
`alertmanager`	prom/alertmanager	9093	Alert routing
`airflow`	apache/airflow	8080	Pipeline orchestration

3. Data Engineering

3.1 Data Source and Collection

I use the `yfinance` Python library to pull historical OHLCV (Open, High, Low, Close, Volume) data for 100 NSE-listed stocks. The universe is selected from the Nifty 100 index — the most liquid large-cap stocks on the National Stock Exchange of India — which ensures sufficient trading history and data quality.

Each stock is pulled with a 3-year lookback to ensure enough data for reliable factor calculation, particularly for longer-window momentum indicators.

3.2 Airflow DAG

Data ingestion is fully automated using Apache Airflow. The DAG (`alphalab_ingestion`) is scheduled to run at 9:00 AM IST on weekdays (after market open). The pipeline is:

1. Pull raw OHLCV data for all 100 tickers
2. Run validation checks
3. Save validated data as Parquet files under `data/raw/`
4. Trigger downstream DVC pipeline via a BashOperator

A separate monthly DAG (`baseline_refresh`) recomputes drift baseline statistics on the first of every month.

4. Feature Engineering

4.1 Factor Philosophy

A "factor" in quant finance is a measurable characteristic of a stock that has been shown to predict future returns. AlphaLab implements four classic factor families, each supported by decades of academic literature:

- **Momentum** — stocks that have gone up recently tend to keep going up (trend following)
- **Value** — cheap stocks (by earnings, book value, etc.) tend to outperform expensive ones
- **Volatility** — lower-volatility stocks tend to produce better risk-adjusted returns
- **Mean Reversion** — stocks that have fallen sharply tend to recover

4.2 Factor Definitions

4.2.1 Momentum Factors

Table 4.1: Momentum factor definitions

Factor	Window	Description
`mom_1m_rank`	21 days	1-month price return, cross-sectionally ranked
`mom_3m_rank`	63 days	3-month price return
`mom_6m_rank`	126 days	6-month price return (primary signal)
`mom_12m_rank`	252 days	12-month price return
`mom_skip_rank`	12m skip 1m	Skip-month momentum (avoids reversal)
`rsi_rank`	14 days	Relative Strength Index
`vol_momentum_rank`	21 days	Volume-weighted momentum

4.2.2 Value Factors

Value factors are approximated from price and volume data since fundamental data (P/E, P/B) is not available through `yfinance` for all NSE stocks. I use price-to-volume ratio as a proxy for valuation.

4.2.3 Volatility Factors

Volatility factors include realised volatility over 21 and 63 day windows, as well as the Garman-Klass estimator which uses OHLC data for a more efficient volatility estimate than close-to-close returns alone.

4.2.4 Mean Reversion Factors

Mean reversion factors capture short-term reversal. I compute z-scores of recent returns relative to a rolling mean, which identifies stocks that have deviated significantly from their recent trend.

4.3 Feature Pipeline

All four factor files are computed by the DVC pipeline and combined into `data/features/all_factors.p` — a matrix of 100 stocks × 21 ranked factor columns. All factor values are cross-sectionally ranked (0 to 1) so that the XGBoost model sees consistent input distributions regardless of market conditions.

4.4 Drift Baseline

As part of monitoring, I compute a baseline statistics file (`baseline_stats.parquet`) containing mean, standard deviation, percentiles (p5, p25, p50, p75, p95), skewness, kurtosis, and null rate for each of the 21 factors over the last 365 days. This baseline is refreshed monthly by the Airflow DAG and used by the drift detection system to identify when factor distributions shift significantly.

5. Model Development

5.1 Problem Framing

I frame stock selection as a *ranking problem*: given today's factor values for 100 stocks, rank them from most likely to outperform to most likely to underperform over the next month. This is a more natural framing than regression (predicting exact returns) because the signal-to-noise ratio in return prediction is very low, but relative ranking is more stable.

5.2 Model: XGBoost Ranker

I use XGBoost with a learning-to-rank objective (`rank:pairwise`). The input is the 21 factor rank columns; the target is the 20-day forward return rank.

```
model = xgb.XGBRanker(
    objective="rank:pairwise",
    n_estimators=200,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
)
```

Listing 5.1: XGBoost model configuration

5.3 Experiment Tracking with MLflow

Every training run is logged to MLflow under two experiments:

- `factor_backtests` — individual factor performance (Sharpe ratio, annual return, max drawdown, win rate)

- `composite_model` — the combined XGBoost model (hyperparameters, feature importances, validation metrics)

Beyond the default autolog, I manually log:

- Factor decay scores (how quickly a factor's predictive power degrades)
- SHAP feature importance values
- The full composite score DataFrame as a CSV artifact
- A confusion matrix of LONG/SHORT/NEUTRAL classification

5.4 Model Registry

The best model from the composite_model experiment is registered in the MLflow Model Registry as `AlphaLab_CompositeModel` and promoted to the `Production` stage. The FastAPI server loads this model at startup by querying the registry for the latest Production version.

5.5 SHAP Explanations

I use the `shap` library to compute per-stock factor attributions. For each stock, SHAP values tell us which of the 21 factors drove its composite score up or down. This is exposed through the `/portfolio` endpoint and displayed in the React frontend as "top 3 factors" for each stock recommendation.

6. Model Deployment

6.1 FastAPI REST API

The model is served through a FastAPI application (`src/api/main.py`). All Parquet result files are loaded once at startup into an in-memory `DataStore` object, avoiding repeated disk I/O on every request.

6.1.1 Endpoints

Table 6.1: FastAPI endpoint specification

Method	Endpoint	Description
GET	`/health`	Liveness probe — always returns 200 if the process is running
GET	`/ready`	Readiness probe — returns 200 only when all data files are loaded
GET	`/metrics`	Prometheus metrics endpoint
GET	`/factors`	Factor leaderboard ranked by Sharpe ratio
GET	`/portfolio`	Top N long + short stocks with SHAP explanations
GET	`/backtest/{factor}`	Full P&L time series for one factor
GET	`/factors/shap`	Global SHAP feature importance
GET	`/stocks/{ticker}`	Full factor profile for a single stock

6.1.2 Prometheus Instrumentation

I use the `prometheus-fastapi-instrumentator` library to automatically expose HTTP metrics:

- `http_requests_total` — request count by endpoint, method, and status code
- `http_request_duration_seconds` — latency histogram (used for p50/p95/p99 computation)

- `http_request_size_bytes` / `http_response_size_bytes` — payload sizes

6.2 Containerisation

The FastAPI application is containerised using a Python 3.12-slim Docker image. The `data/` directory is mounted as a read-only volume so that updated Parquet files from the DVC pipeline are picked up without rebuilding the image.

```
# Build the image
docker-compose build api

# Start the full stack
docker-compose up -d

# Check API logs
docker-compose logs api --tail=20
```

Listing 6.1: Docker build and run commands

7. Monitoring and Observability

7.1 Prometheus Setup

Prometheus is configured to scrape the FastAPI `/metrics` endpoint every 15 seconds. The configuration file (`prometheus.yml`) defines four scrape jobs: `alphalab_api`, `airflow`, `node`, and `prometheus` itself.

7.2 Grafana Dashboard

The Grafana dashboard is fully provisioned via configuration files — no manual setup is required. It consists of 13 panels organised into five sections:

Table 7.1: Grafana dashboard panels

Section	Panel	Metric
API Health	API Status	`up{job="alphalab_api"}`
	Request Rate	`rate(http_requests_total[5m])`
	Error Rate (5xx)	`rate(5xx) / rate(total)`
	p95 Latency	`histogram_quantile(0.95, ...)`
	Predictions (24h)	`increase(predictions_total[24h])`
Request Metrics	Rate by Endpoint	Per-endpoint breakdown
	Latency Percentiles	p50, p95, p99 time series
Factor Performance	Factor Scores	`factor_score` gauge
	Drift PSI Score	`drift_detection_psi_score`
Infrastructure	CPU Usage	`node_cpu_seconds_total`
	Memory Usage	`node_memory_MemAvailable_bytes`
	Disk Usage	`node_filesystem_avail_bytes`
Active Alerts	Alert table	`ALERTS{job="alphalab_api"}`

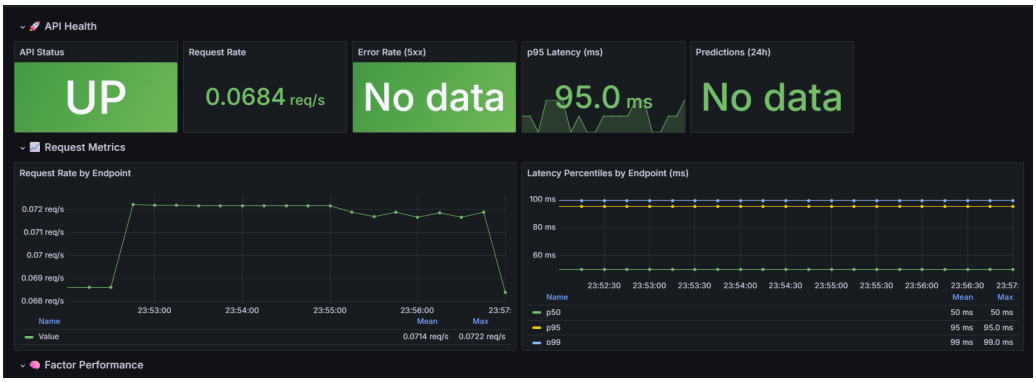

Figure 7.1: AlphaLab Grafana dashboard

7.3 Alert Rules

I configured the following Prometheus alert rules in `alerts/alphalab_alerts.yml`:

Table 7.2: Prometheus alert rules

Alert	Condition	Duration	Severity
HighErrorRate	5xx rate $> 5\%$	2 min	Critical
HighLatencyP95	p95 > 500ms	3 min	Warning
CriticalLatencyP95	p95 > 1000ms	2 min	Critical
APIDown	`up == 0`	2 min	Critical
FactorDriftDetected	PSI score > 0.2	Immediate	Warning
HighCPUUsage	CPU $> 85\%$	5 min	Warning
LowDiskSpace	Disk $< 15\%$ free	5 min	Warning

7.4 Drift Detection

I compute Population Stability Index (PSI) as the primary drift metric. PSI measures how much the distribution of a feature has shifted between a reference period (the baseline) and the current period. A PSI value above 0.2 indicates significant drift and triggers the `FactorDriftDetected` alert.

The drift baseline is computed by `src/monitoring/calculate_baseline.py` over the last 365 days of data and stores 16 statistics per feature. It is refreshed monthly by the Airflow `baseline_refresh` DAG.

8. Frontend

8.1 Overview

The frontend is a React single-page application that communicates with the FastAPI backend exclusively through REST API calls.

8.2 Application Tabs

8.2.1 Tab 1: Factor Leaderboard

This view shows all computed factors ranked by their Sharpe ratio from backtesting. Users can filter by factor status (active, decaying, weak) and see key metrics: annual return, maximum drawdown, win rate, and factor decay score. This helps a user quickly identify which factors are currently working in the market.

8.2.2 Tab 2: Factor Deep Dive

Clicking on any factor opens a detailed view with its full P&L time series chart, showing cumulative performance over the backtest period. This allows the user to visually assess not just the final Sharpe ratio but also whether the factor has been consistently profitable or has had long drawdown periods.

A global SHAP importance bar chart shows which factors contribute most to the composite model's predictions.

8.2.3 Tab 3: Portfolio View

This is the main output view. It shows the top 10 long recommendations (stocks to buy) and top 10 short recommendations (stocks to sell or avoid), each with their composite alpha score, predicted return, and the top 3 factor drivers from SHAP analysis.

Clicking on any stock row shows its full factor profile — all 21 raw factor values plus the SHAP attribution breakdown.

8.3 API Communication

The frontend fetches data from the following endpoints on load and on user interaction:

```
// Tab 1: Factor leaderboard
const factors = await fetch('http://localhost:8000/factors?limit=25')

// Tab 2: Factor deep-dive
const backtest = await fetch(`http://localhost:8000/backtest/${
    factorName}`)
const shap = await fetch('http://localhost:8000/factors/shap')

// Tab 3: Portfolio
const portfolio = await fetch('http://localhost:8000/portfolio?top_n
    =10')

// Stock detail
const stock = await fetch(`http://localhost:8000/stocks/${ticker}`)
```

Listing 8.1: Frontend API calls

9. Testing

9.1 Unit Tests

I implemented a test suite in `src/tests/` covering six test files:

Table 9.1: Test suite summary

File	Tests	Coverage
`test_momentum.py`	6	Momentum factor computation
`test_value.py`	4	Value factor computation
`test_volatility.py`	5	Volatility factor computation
`test_mean_reversion.py`	5	Mean reversion factor computation
`test_combine_factors.py`	4	Factor combination and ranking
`test_alphalab_ci.py`	16	Data validation, baseline, API, smoke
Total	**40**	

9.2 Test Categories

- **Feature tests** — verify that computed factor values fall within expected ranges and have acceptable null rates
- **Data validation tests** — verify that the 8 ingestion checks catch known bad inputs
- **Monitoring tests** — verify that `calculate_baseline` produces correct statistics on synthetic data
- **Smoke tests** — verify all required packages are importable and DVC is initialised

9.3 Running Tests

```
# From project root
pytest src/tests/ -v --tb=short

```

```
# With coverage report
pytest src/tests/ --cov=src --cov-report=term-missing
```

Listing 9.1: Running the test suite

10. Known Gaps and Future Work

10.1 Known Gaps

Being honest about what I did not finish is important. The following items are either partially implemented or not implemented:

- **Feedback loop** — the system does not yet log actual forward returns when they become available and compare them to predictions. This would close the loop and enable true online model evaluation.
- **Rollback mechanism** — while multiple model versions exist in the MLflow registry, there is no automated rollback script that would revert to a previous version if the current one degrades.
- **Data encryption** — data is stored unencrypted at rest. For a production system handling real financial data, encryption at rest and in transit would be mandatory.
- **Node exporter** — the infrastructure panels in Grafana (CPU, memory, disk) require a `node-exporter` container which is not yet added to Docker Compose.
- **Model quantisation** — XGBoost models do not benefit meaningfully from quantisation, so this was intentionally skipped.

10.2 Future Work

- Expand the stock universe to the full Nifty 500
- Add fundamental data (P/E, P/B, ROE) via a financial data provider
- Implement online learning to update factor weights daily
- Add a paper trading mode that tracks hypothetical P&L in real time
- Deploy to a cloud provider (AWS/GCP) with a managed PostgreSQL backend for MLflow

11. Conclusion

AAlphaLab was an attempt to build an end-to-end quantitative research system. The project highlighted the gap between developing models and running them reliably in production, with most challenges coming from system integration (Docker, Prometheus, Grafana).

The factors performed reasonably well for a prototype, with momentum showing the strongest results. Combining factors using XGBoost improved performance over individual signals.

Future work would focus on adding a feedback loop to track real outcomes and improve monitoring.

A. Project Structure

```
Alphalab/
          dags/                    # Airflow DAG definitions
                alphalab_ingestion.py
                baseline_refresh_dag.py
          data/
                raw/                 # Raw OHLCV parquets (DVC tracked)
                features/            # Factor parquets + baseline stats
                backtest/            # Factor P&L and rankings
                models/              # Composite scores + SHAP
          grafana/
                provisioning/        # Datasource + dashboard JSON
          src/
                api/
                      main.py          # FastAPI application
                factors/             # Factor computation modules
                models/              # XGBoost training
                monitoring/          # Baseline + drift detection
                tests/               # pytest test suite
          alerts/
                alphalab_alerts.yml  # Prometheus alert rules
          Dockerfile               # API container
          docker-compose.yml       # Full stack orchestration
          prometheus.yml           # Prometheus configuration
          alertmanager.yml         # Alertmanager configuration
          dvc.yaml                 # DVC pipeline definition
          requirements.txt
```

Listing A.1: AlphaLab directory structure